

Looping

Chapter 5

Copyright © 2017 STEM Course Prep



WHAT ABOUT THIS PROBLEM?

$$\sum_{i=1}^n x^i \quad \text{where } n = \text{user input}$$

- Think about what is going on in this summation....
$$x^1 + x^2 + x^3 + \dots + x^n$$
- How would you code this in Matlab if n can change or if n is very large?

Answer: a **for** loop

WHY LOOP STATEMENTS?

- Imagine having one equation that needed to be calculated over and over again
- With a loop statement there's no need to duplicate the equation in the script
 - MATLAB will repeat the calculation n-times.
- Loops are for repetitive tasks
- Two kinds of loop statements in MATLAB
 - for
 - while

for LOOP

- Also known as a counted loop
- Repeats an action a specified number of times
 - You must know ahead of time how many times the action needs to be done
- A loop variable (or iterator) will be used to count the loops

for LOOP

```
for loopVar = range  
    Action(s)  
end
```

for 'for' is a reserved word in MATLAB

loopVar The loop iterator variable. Typically a single letter like i, j, k but can be any valid variable name

range Range of values for the loop iterator variable. Any vector can be used. Colon operator is typically used

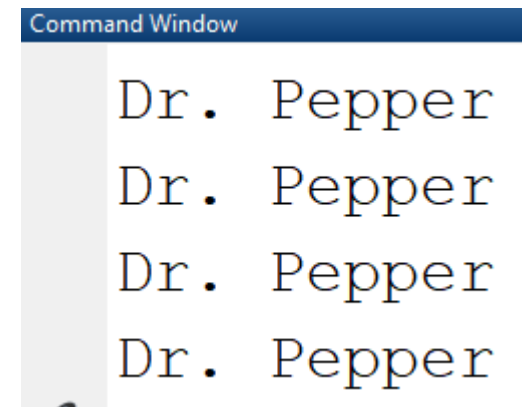
Action(s) Actions/calculations to be done for each iteration

end end – code goes back to the beginning of the loop if iterations remain. Otherwise it goes to the next line of code.

for LOOP EXAMPLES

- Repeating a phrase:

```
for i=1:4  
    disp('Dr. Pepper')  
end
```



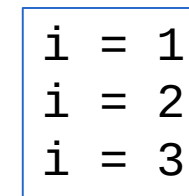
Command Window

```
Dr. Pepper  
Dr. Pepper  
Dr. Pepper  
Dr. Pepper
```

A screenshot of the MATLAB Command Window. The title bar is blue and says "Command Window". The window contains four lines of text, each "Dr. Pepper", stacked vertically. A red arrow points from the `disp('Dr. Pepper')` line in the code block to this window.

- See how the iterator changes

```
for i=1:3  
    i  
end
```



```
i = 1  
i = 2  
i = 3
```

A screenshot of the MATLAB Command Window showing the values of the iterator `i`. The window contains three lines of text: `i = 1`, `i = 2`, and `i = 3`, stacked vertically. A red arrow points from the `i` line in the code block to this window.

for LOOP EXAMPLES

- Want to use a different step size? Use the colon operator

```
for k=3:3:12  
    fprintf('k = %d\n', k)
```

```
end
```

```
k = 3
```

```
k = 6
```

```
k = 9
```

```
k = 12
```

FOR LOOP EXAMPLES

- Loop variable can have positive or negative values

```
for j=10:-2:-4
    count=j+10;
    fprintf('j= %d and count= %d\n',j,count)
end
```

```
j= 10 and count= 20
j= 8 and count= 18
j= 6 and count= 16
j= 4 and count= 14
j= 2 and count= 12
j= 0 and count= 10
j= -2 and count= 8
j= -4 and count= 6
```

FOR LOOP EXAMPLES

- Can use loop variable after loop is complete

```
for time=1:0.5:5
    velocity=10+2*time;
    fprintf(' At time= %.2f velocity = %.2f\n',time,velocity)
end
s=3+10*time+.5*2*time^2;
fprintf ('After %.2f seconds the displacement = %.2f\n',time,s)
```

```
At time= 1.00 velocity = 12.00
At time= 1.50 velocity = 13.00
At time= 2.00 velocity = 14.00
At time= 2.50 velocity = 15.00
At time= 3.00 velocity = 16.00
At time= 3.50 velocity = 17.00
At time= 4.00 velocity = 18.00
At time= 4.50 velocity = 19.00
At time= 5.00 velocity = 20.00
After 5.00 seconds the displacement = 78.00
```

PREALLOCATING A VECTOR

- Say we need to prompt the user for information a number of times(N)

```
for i = 1:N
    % prompt user for input
    % actions
end
```

Note: this is not an optimal way of doing things – this is to demonstrate preallocation

- Entered values need to be stored in a vector
- Two ways of doing this:
 - Append data to the vector as input is provided by the user
 - Preallocate the vector

PREALLOCATING A VECTOR

- Preallocating – basically you create the vector in the size you need with placeholder values.
 - This sets aside needed memory which saves time and makes things more efficient
- Extending a vector is inefficient
- Common to use the **zeros** function to preallocate a vector
- For example, preallocate a matrix *mat* to have $n \times n$ elements:

```
mat = zeros(n);
```

PREALLOCATING A VECTOR

- Check the efficiency of your code by using:
 - **tic** and **toc**
- **tic** and **toc** provide a way to time how long it takes to run code
- **tic** – starts the timer
- **toc** – stops the timer and provides the run time

```
tic;  
test=rand(1000,3);  
toc;
```

```
>> in_class_chapter_5  
Elapsed time is 0.000089 seconds.
```

PREALLOCATING A VECTOR

```
clear all
tic;
temp_C=randi([0,100],1000,1);
%temp_K=zeros(1000,1);
for i=1:1000
    temp_K(i)=temp_C(i)+273.15;
end
toc;
```

→ preallocation

No preallocation →

```
>> in_class_chapter_5
Elapsed time is 0.001285
seconds.
```

Preallocation →

```
>> in_class_chapter_5
Elapsed time is 0.001278
seconds.
```

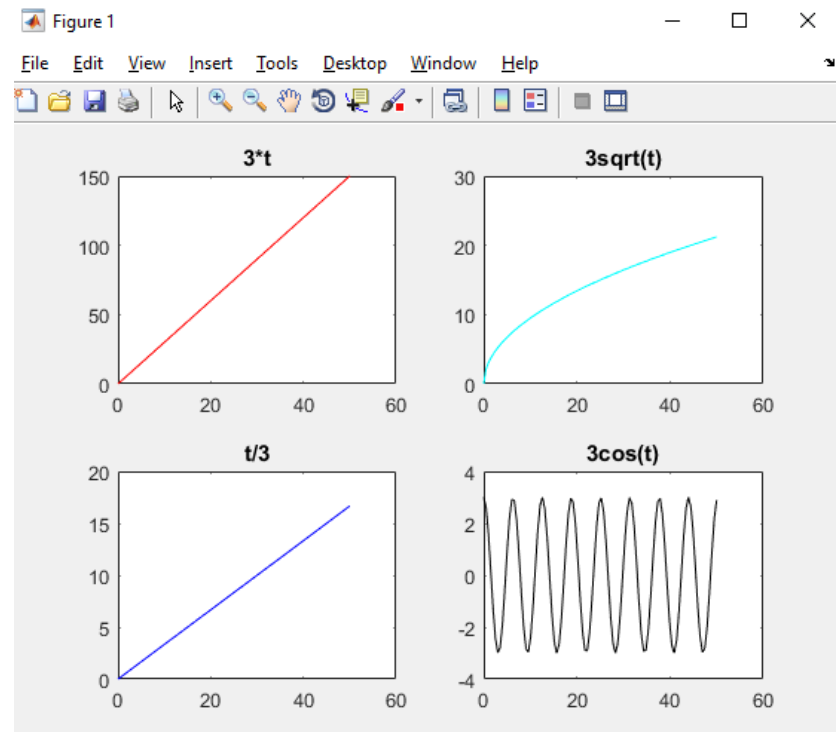
subplot

- **subplot** creates a matrix of plots
- Syntax:
 - **subplot(m,n,i)** gives an mxn matrix of plots
 - i gives the number of the individual plot
 - Plots are numbered row-wise
 - Example of numbering for a 2x3 matrix of plots

i=1	i=2	i=3
i=4	i=5	i=6

subplot EXAMPLE

```
t=linspace(0,50);  
subplot(2,2,1)  
plot(t,3*t,'r')  
title('3*t')  
subplot(2,2,2)  
plot(t,3*sqrt(t),'c')  
title('3sqrt(t)')  
subplot(2,2,3)  
plot(t,t/3,'b')  
title('t/3')  
subplot(2,2,4)  
plot(t,3*cos(t),'k')  
title('3cos(t)')
```



subplot WITH for LOOPS

- Can use for loops to create the subplot commands

```
for i = 1:4  
    subplot(2,2,i)  
    % create plot i  
end
```



subplot(2,2,1)	subplot(2,2,2)
subplot(2,2,3)	subplot(2,2,4)

COMBINING **for** AND **if**

- Options for combining **for** loops and **if** statements

- **if** statement goes in the loop

- Each time a loop is done the **if** statement is checked

```
for i=1:4
    if i<2
        j=i+1
    end
end
```

- **if** statement's action includes a **for** loop

```
if a==10
    for j=1:2
        k=j*j
    end
end
```

NESTED for LOOPS

- A nested **for** loop: one loop is inside another
- General form of a nested **for** loop:

```
for loopVar1 = range1
    action(s)
    for loopVar2 = range2
        action(s)
    end
    action(s)
end
```

Outer loop

Inner loop

- Inner loop is fully completed before the outer loop increments

NESTED FOR LOOPS

```
for i=1:3
    for j=1:2
        fprintf('i= %d    j= %d    \n',i,j)
    end
end
```

i= 1	j= 1		Notice that i does not change until j has gone through its range of values
i= 1	j= 2		
i= 2	j= 1		
i= 2	j= 2		
i= 3	j= 1		
i= 3	j= 2		

Nested loops are useful in
creating a matrix of data

WHILE LOOP

- **for** loop is a counted loop
 - You know how many iterations you need
- **while** loops are conditional
 - Looping will continue until a certain condition is met
 - Counter variables can be used to count the number of iterations
- Note: check your code to make sure you actually can enter the while loop

while LOOP

```
while condition
    action(s)
end
```

while is a reserved word in MATLAB

Logical statement to be tested at the beginning of each iteration.
Logical = TRUE, loop continues. Logical = FALSE, loop ends.

Actions/calculations to be performed if logical is TRUE

Ends the current loop. Goes back to the beginning and retests the logical.

If TRUE, actions in the while loop are performed.

If FALSE, goes to the line of code after the end statement.

EXAMPLE OF A `while` LOOP

- `i` will be increased by 3 as long as the condition `i<20` is met
- Once `i>20`, the code following the `end` statement will be executed

```
%while loop example
clear all
i=0;
while i<20
    fprintf('i= %d\n',i)
    i=i+3;
end
fprintf('Done\n')
```

```
>> ch_5_slides
i= 0
i= 3
i= 6
i= 9
i= 12
i= 15
i= 18
Done
```

COUNTING IN A WHILE LOOP

- Many times we want to count the number of times the while loop has looped
 - Many numerical techniques include a check for a max number of iterations – a **counter** is useful for this
- Setting up a counter:

```
count=0;  
i=0;  
while i<20  
    fprintf('i= %d\n',i)  
    i=i+3;  
    count=count+1;  
end  
fprintf('Count=%d\n',count)
```



```
i= 0  
i= 3  
i= 6  
i= 9  
i= 12  
i= 15  
i= 18  
Count=7
```

CAUTION!

- Infinite loops are possible with `while` loops
- What is an infinite loop?
 - A loop that will go on indefinitely
- Cause: the logical tested never becomes FALSE
- How to end it:
 - CTRL+C (might not work on the first attempt)
- Using a counter is one way to avoid these
 - Once it goes past an excessive number of iterations the loop will end if a counter is included in the logical

CAUTION!

- What's wrong with this code?
- When it runs it results in countless `i=0` statements.
- This loop will never end because `i` will always be `<20`
- CTRL+C was used to get out of loop.

```
%while loop example
clear all
i=0;
while i<20
    fprintf('i= %d\n',i)
end
fprintf('Done\n')
```

```
i= 0
i= 0
i= 0
i= 0
i= 0
i= 0
```

Operation terminated by user
during ch_5_slides (line 114)

ERROR CHECKING

- Can use while loops to check for user's input errors
- Use logicals to ensure the input is a valid entry

```
radius = input('Enter the radius of a circle: ');  
while radius <= 0  
    radius = input('Invalid! Enter a positive radius: ');  
end  
area = pi * radius ^ 2;  
fprintf('The area is %.2f\n', area)
```

```
>> ch_5_slides  
Enter the radius of a circle: -10  
Invalid! Enter a positive radius: 2  
The area is 12.57
```

VECTORIZING

- MATLAB is designed for use with matrices
 - It has many built in matrix functions
 - Other programming languages do not have these built in functions so they require loops
- **Vectorizing** - re-writing code to use the matrix operations in MATLAB instead of loops
- For example, instead of having to use nested loops to multiply matrices, you can just do $C=A*B$
- Best to avoid loops in MATLAB when possible – VECTORIZE!

EFFICIENCY

- Built-in functions produce more efficient code:
 - **sum, prod, cumsum, cumprod, min, max**
 - **any, all, find**
 - **diff**
- **checkcode**: checks code in scripts and functions for inefficiencies

```
for i=1:4  
    vector(i)=i*2  
end
```

```
>>checkcode inefficient
```

L 2 (C 5-10): The variable 'vector' appears to change size on every loop iteration (within a script). Consider preallocating for speed.

L 2 (C 14): Terminate statement with semicolon to suppress output (within a script).